

SQL-Based Data Analysis Report

Celebal Technologies Internship

Assignment No-2: Week 2 SQL Assignment

Database: Ecommerce_sales

Author: Snehal A. Bhosale

College: Sanjivani College OF Engineering, Kopargaon 423603

E-mail: snehalbhosale1807@gmail.com

Contents

Sr. No	Topic	Page No.
1.	Project Overview	02
2.	Objective	02
3.	Tools	02
4.	Database Creation	02
5.	Table Creation	02
6.	Index Creation	03
7.	Data Insertion	03
8.	Data Validation	03
9.	Section A	03
10.	Section B	09
11.	Section C	16
12.	Section D	25
13.	Section E	32
14.	Business Insights	38
15.	Learning Outcomes	38
16.	Conclusion	39
17.	GitHub Repository	39

1. Project Overview

This project focuses on performing SQL-based data analysis on the **Ecommerce_sales** database using **MySQL Workbench**. The dataset was explored to understand its structure and relationships before applying various SQL operations. Different queries involving filtering, aggregation, joins, conditional logic, and transaction management were executed to extract meaningful business information. Query optimization techniques such as indexing and efficient filtering were also considered to improve performance. The analysis demonstrates how SQL can be used to transform raw transactional data into valuable insights that support business decision-making.

2. Objective

- To analyze e-commerce sales data using SQL and extract meaningful business insights.
- To apply SQL concepts such as filtering, aggregation, joins, CASE statements, and transactions on a relational database.
- To write optimized, well-structured, and professional SQL queries following best practices.
- To generate accurate reports that support data-driven business analysis and decision-making.

3. Tools Used

- MySQL Workbench
MySQL
GitHub
Microsoft Word

4. Database Creation:

- **Description:** Create and select the Ecommerce_sales database.
- **SQL Query:**

```
CREATE DATABASE Ecommerce_sales ;  
USE Ecommerce_sales;
```

5. Table Creation:

The required database tables were created according to the provided schema using appropriate SQL CREATE TABLE statements. Primary keys, foreign keys, and necessary constraints were defined to establish relationships and maintain data integrity across the database.

Index Creation

6. Create indexes for performance optimization:

Indexes were created on frequently queried columns to improve database performance. These indexes help optimize filtering, sorting, and search operations, resulting in faster query execution on large datasets.

7. Data Insertion:

The provided sample dataset was successfully inserted into all database tables using SQL INSERT statements. The insertion process ensured that all related records were loaded while preserving referential integrity.

8. Data Validation:

Data validation was performed by checking the total number of records in each table and reviewing sample records using SELECT queries. This confirmed that the data was imported correctly and that the database was ready for analysis.

9. Section A – SQL Basics (SELECT, Constraints, Primary Keys)

Overview: This section focuses on the fundamental SQL operations required to explore and validate the database. It covers retrieving data using SELECT statements, understanding primary keys and constraints, and verifying data integrity through practical examples. These concepts form the foundation for performing reliable SQL-based data analysis.

Question 1. Write a query to display all columns and rows from the customer's table.

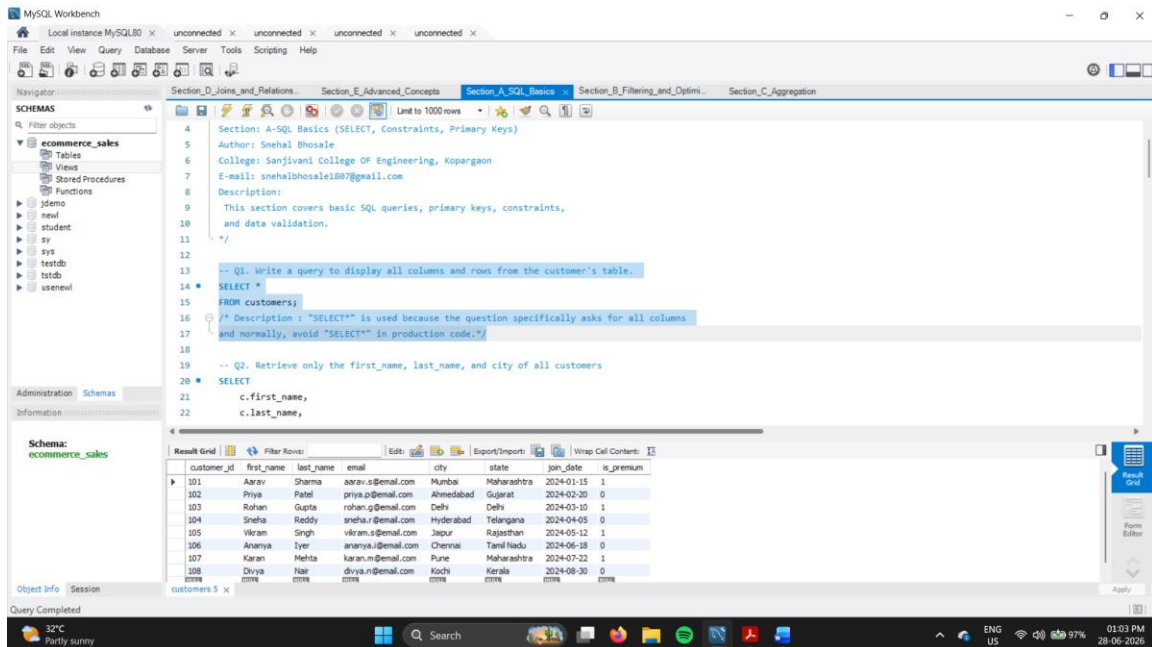
Answer:

- **SQL Query:**

```
SELECT *
```

```
FROM customers;
```

- **Description:** This query retrieves every record and every column from the customers table. It is useful for viewing the complete dataset during initial database exploration. Although SELECT * is acceptable for this task, selecting only the required columns is considered a better practice in real-world applications.
- **Output:**



Question 2. Retrieve only the first_name, last_name, and city of all customers

Answer:

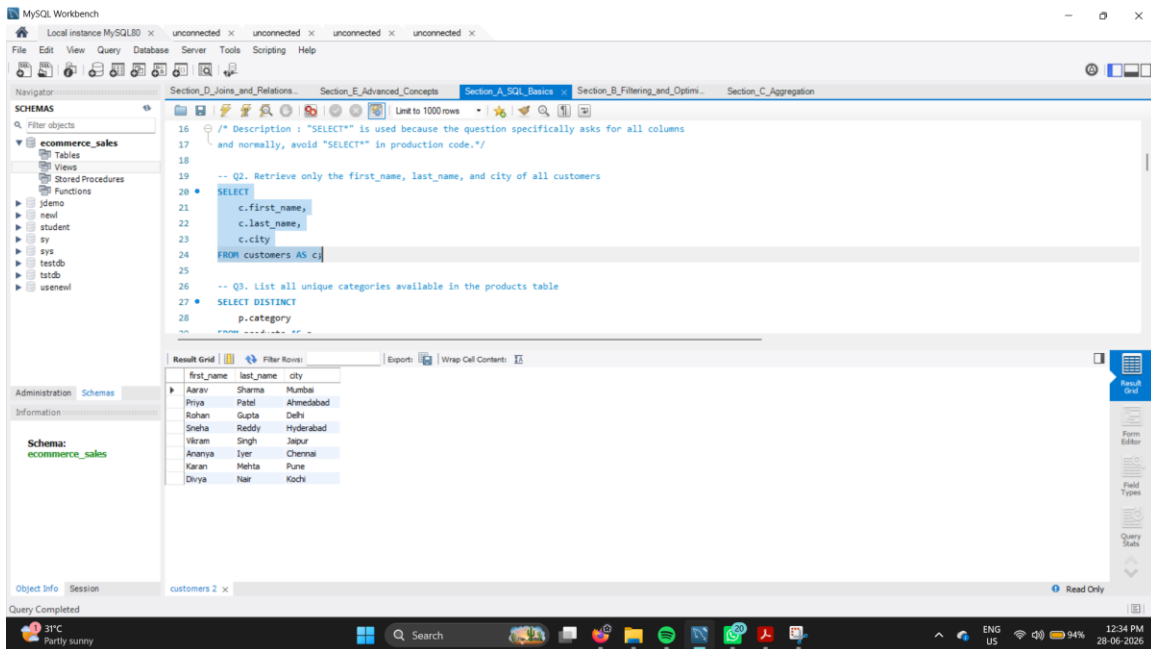
- **Query:**

```

SELECT
    c.first_name,
    c.last_name,
    c.city
  
```

FROM customers AS c;

- **Description:** This query displays only the required customer information instead of retrieving every column. Using a table alias (c) improves readability and follows professional SQL coding standards.
- **Output:**



Question 3. List all unique categories available in the products table

Answer:

- Query:

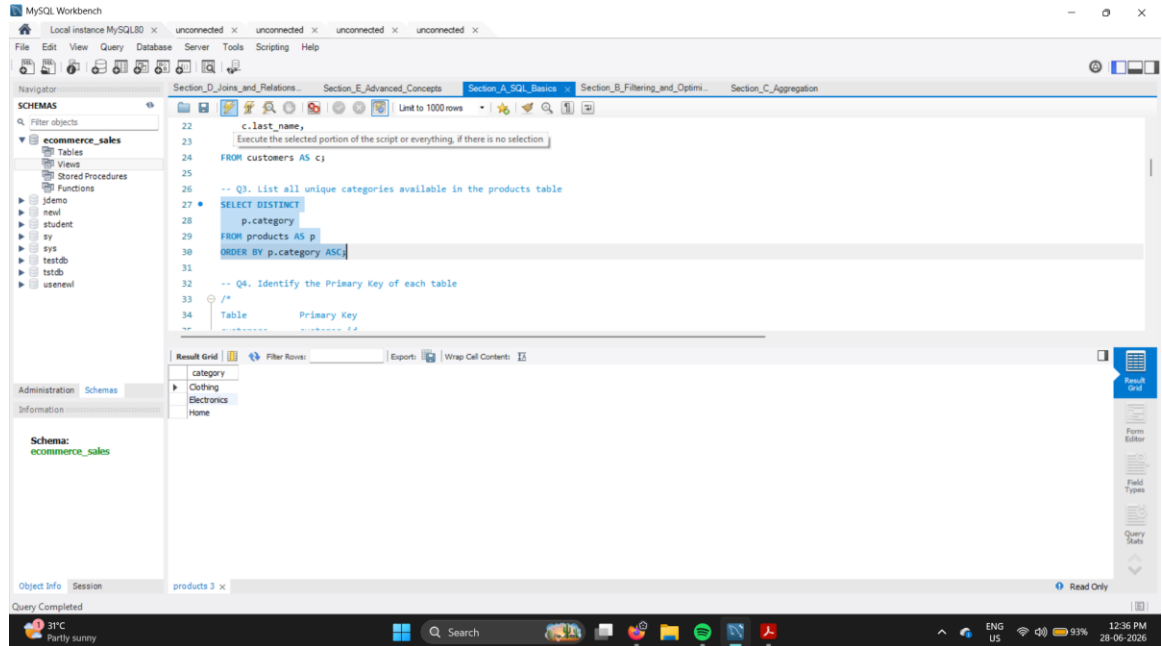
```
SELECT DISTINCT
```

```
p.category
```

```
FROM products AS p
```

```
ORDER BY p.category ASC;
```

- **Description:** The DISTINCT keyword removes duplicate category names, ensuring that each product category appears only once. The results are arranged alphabetically using the ORDER BY clause.
- **Output:**



Question 4. Identify the Primary Key of each table

- **Answer:**

Table Name	Primary Key
customers	customer_id
products	product_id
orders	order_id
order_items	item_id

- **Description:** A primary key uniquely identifies each record in a table. It ensures that every row has a unique identifier, prevents duplicate records, and supports relationships between tables through foreign keys.

Question 5. Constraints on the email column

Answer:

The email column contains the following constraints:

- **UNIQUE**
- **NOT NULL**
- **Demonstration Query**

```

INSERT INTO customers
(
  customer_id,
  first_name,
  last_name,

```

```

email,
city,
state,
join_date,
is_premium
)
VALUES
(
109,
'Rahul',
'Patil',
'aarav.s@email.com',
'Nagpur',
'Maharashtra',
'2024-09-01',
TRUE
);

```

- **Description:** The NOT NULL constraint ensures that every customer record must contain an email address, while the UNIQUE constraint prevents duplicate email entries. Attempting to insert an existing email address results in a database error, maintaining data integrity.

Question 6. Insert a product with a negative price.

Answer:

- **SQL Query**

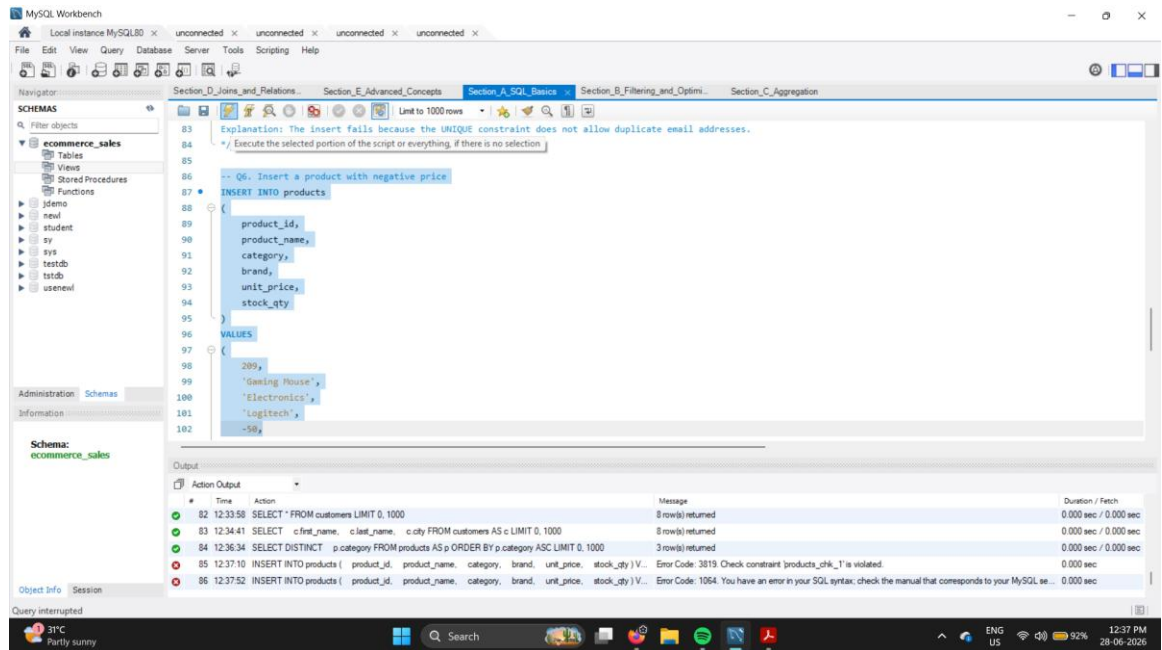
```

INSERT INTO products
(
product_id,
product_name,
category,
brand,
unit_price,
stock_qty
)
VALUES
(
209,
'Gaming Mouse',
'Electronics',
'Logitech',
-50,
20

```

);

- **Description:** This query is used to verify the CHECK constraint defined on the unit_price column. Since product prices cannot be negative, MySQL rejects the insertion and protects the database from invalid values.
- **Output:**



Display the total number of records available in each table.

- **SQL Query**
`SELECT 'Customers' AS table_name, COUNT(*) AS total_records
FROM customers`

`UNION ALL`

`SELECT 'Products', COUNT(*)
FROM products`

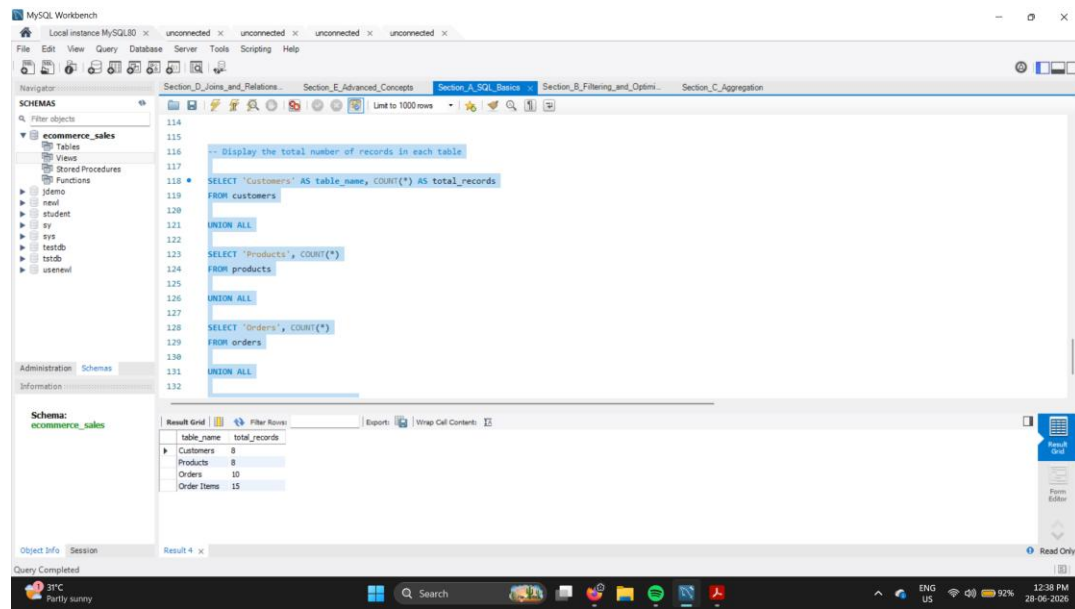
`UNION ALL`

`SELECT 'Orders', COUNT(*)
FROM orders`

`UNION ALL`

`SELECT 'Order Items', COUNT(*)
FROM order_items;`

- **Description:** This validation query counts the total number of records present in each table. It is commonly executed after data insertion to verify that all records have been successfully loaded into the database.
- **Output:**



10. Section B – Filtering & Query Optimization

Overview: This section demonstrates the use of filtering techniques to retrieve specific records from the database using the WHERE clause, logical operators, and date ranges. It also explains how indexes improve query performance and highlights best practices for writing optimized SQL queries.

Question 7. Retrieve all orders with the status 'Delivered'.

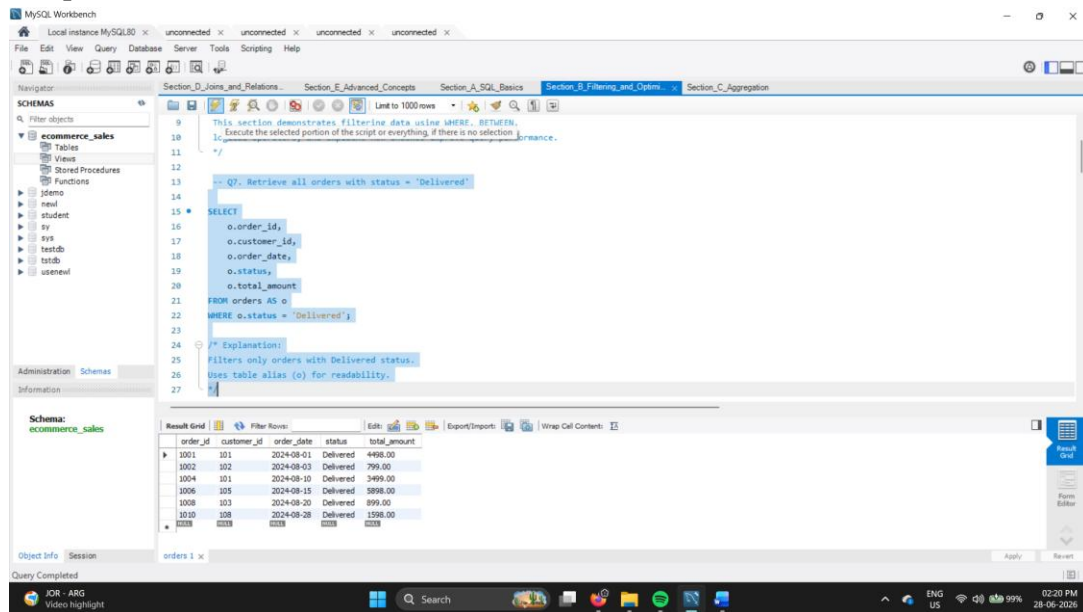
Answer:

- **SQL Query**

```
SELECT
    o.order_id,
    o.customer_id,
    o.order_date,
    o.status,
    o.total_amount
```

```
FROM orders AS o
WHERE o.status = 'Delivered';
```

- **Description:** This query filters the orders table to display only those orders whose status is Delivered. The use of the table alias (o) improves query readability and follows professional SQL coding standards.
- **Output:**



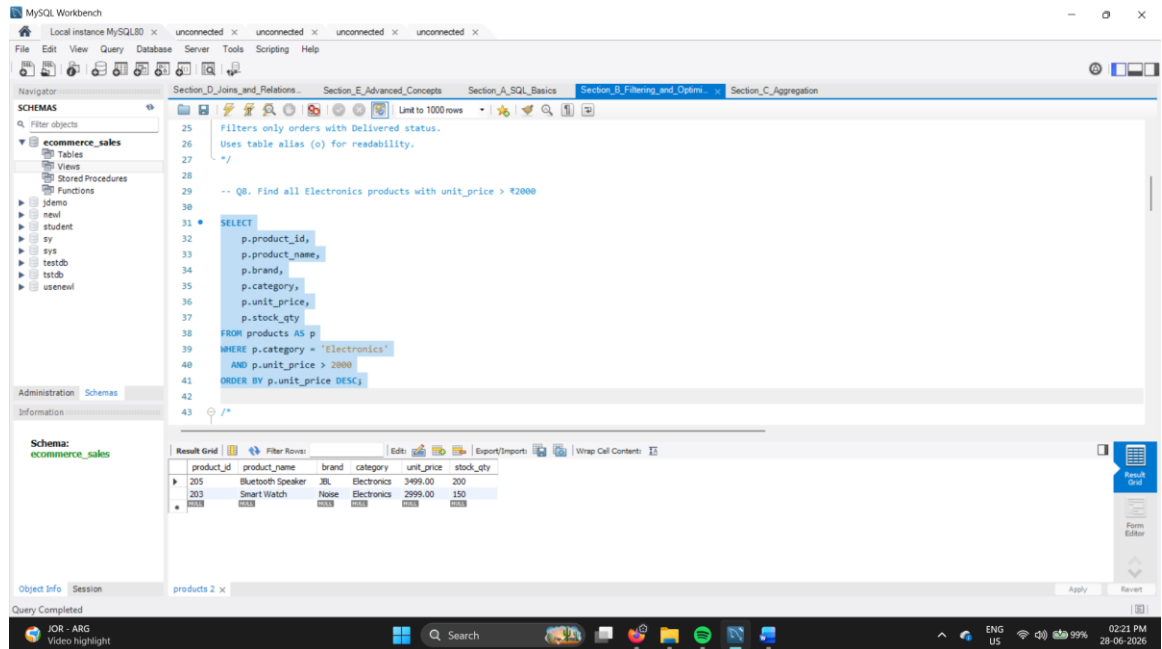
Question 8. Find all Electronics products with a unit price greater than ₹2000.

Answer:

- **SQL Query:**

```
SELECT
  p.product_id,
  p.product_name,
  p.brand,
  p.category,
  p.unit_price,
  p.stock_qty
FROM products AS p
WHERE p.category = 'Electronics'
  AND p.unit_price > 2000
ORDER BY p.unit_price DESC;
```

- **Description:** This query retrieves products that belong to the Electronics category and have a price greater than ₹2000. Multiple conditions are combined using the AND operator, and the results are sorted in descending order of product price.
- **Output:**

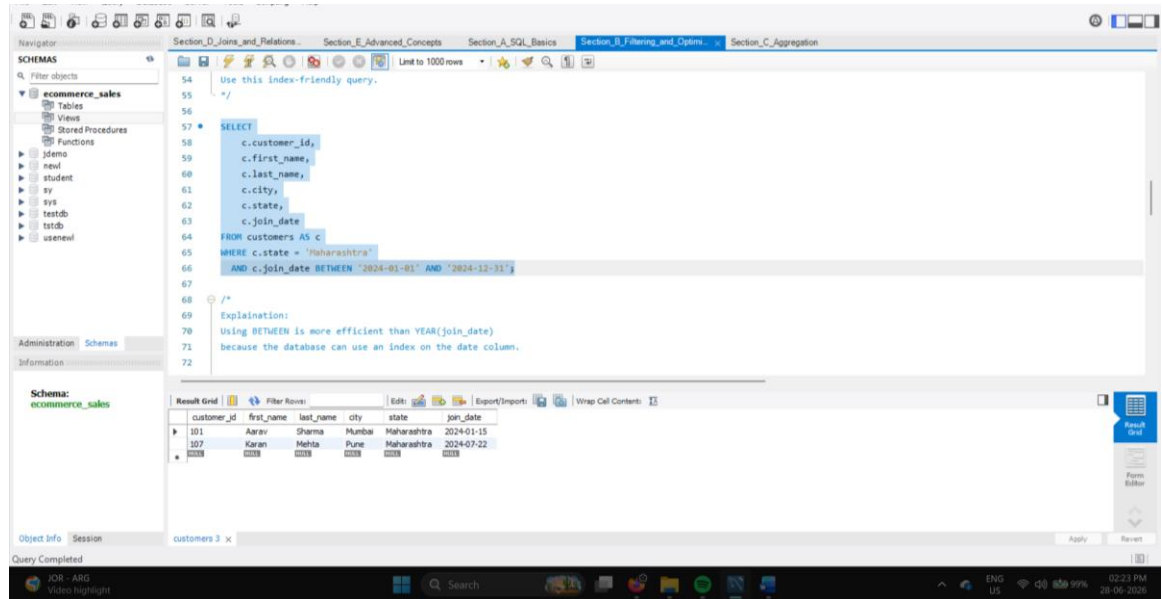


Question 9. Display customers from Maharashtra who joined during 2024.

Answer:

- **SQL Query:**

```
SELECT
  c.customer_id,
  c.first_name,
  c.last_name,
  c.city,
  c.state,
  c.join_date
FROM customers AS c
WHERE c.state = 'Maharashtra'
  AND c.join_date BETWEEN '2024-01-01' AND '2024-12-31';
```
- **Description:** This query filters customers based on their state and joining date. The BETWEEN operator is used to retrieve records within a specified date range. This method is more efficient than using the YEAR() function because it allows MySQL to utilize indexes.
- **Output:**



Question 10. Display all orders placed between 10-Aug-2024 and 25-Aug-2024, excluding cancelled orders.

Answer:

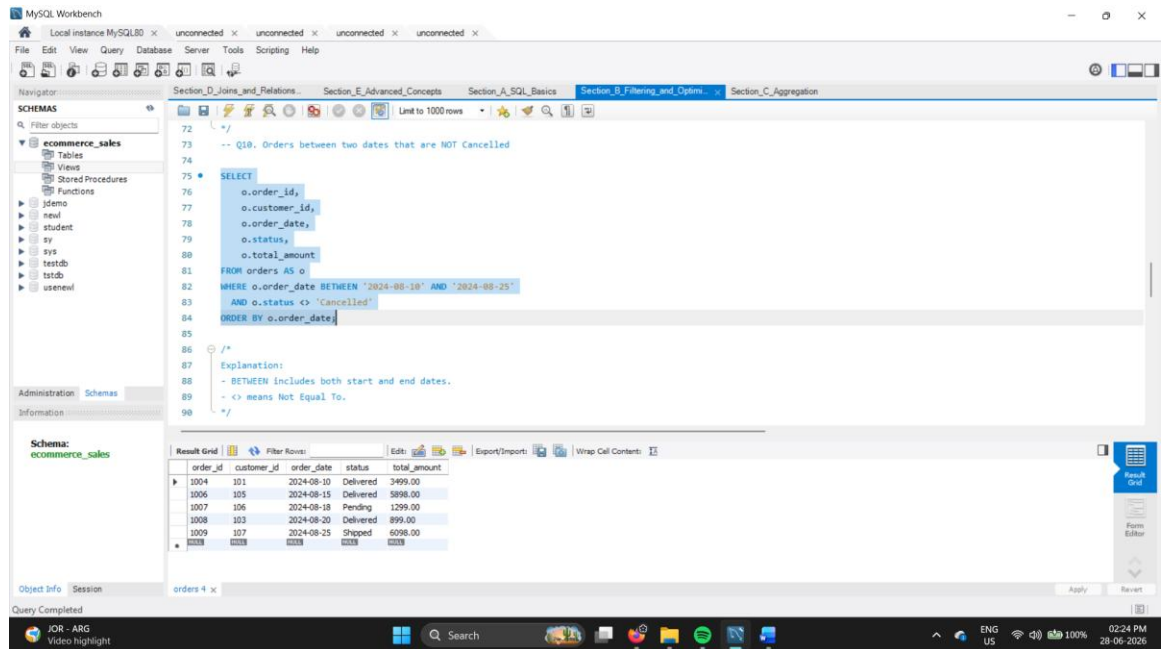
- **SQL Query**

```

SELECT
    o.order_id,
    o.customer_id,
    o.order_date,
    o.status,
    o.total_amount
FROM orders AS o
WHERE o.order_date BETWEEN '2024-08-10' AND '2024-08-25'
    AND o.status <> 'Cancelled'
ORDER BY o.order_date;

```

- **Description:** This query retrieves orders placed within the specified date range while excluding cancelled orders. The results are arranged by order date, making them easier to analyze chronologically.
- **Output:**



Question 11. Explain the purpose of the **idx_orders_date** index.

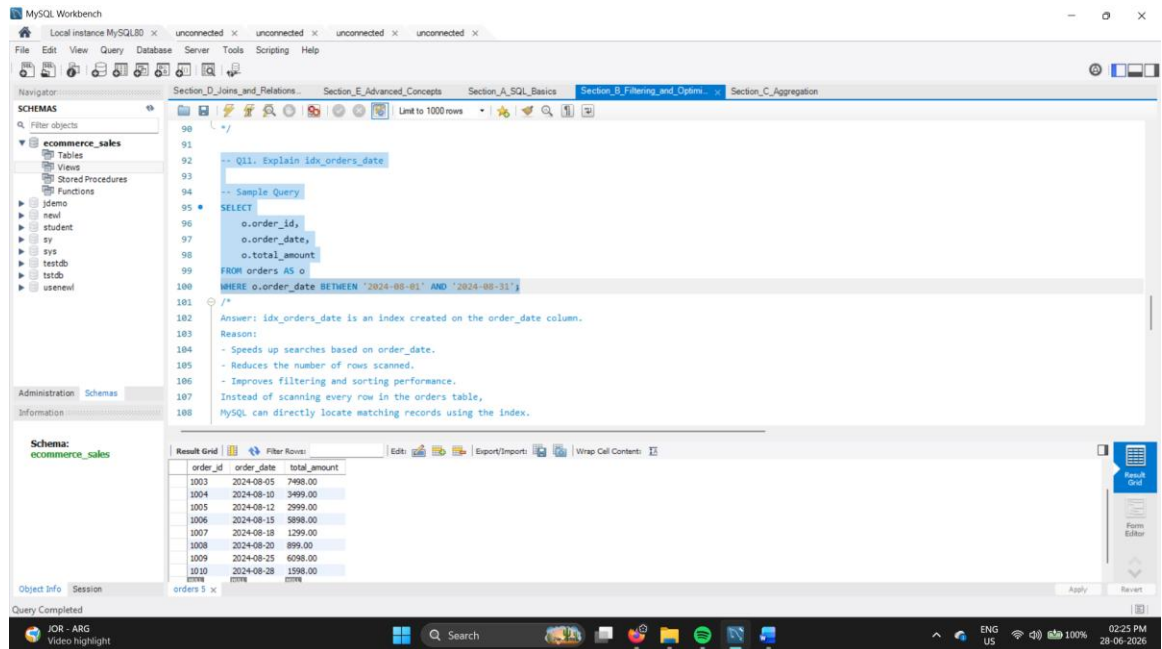
Answer: The **idx_orders_date** index is created on the **order_date** column to improve the performance of queries that search or sort data based on order dates.

- **Description:** Indexes help MySQL locate matching records without scanning the entire table. As a result, queries execute faster, especially when working with large datasets.
- **Demonstration Query**

```

SELECT
    o.order_id,
    o.order_date,
    o.total_amount
FROM orders AS o
WHERE o.order_date BETWEEN '2024-08-01' AND '2024-08-31';

```
- **Output:**



Question 12. Is YEAR(join_date) an index-friendly approach?

Answer: No. Applying the YEAR() function to an indexed column prevents MySQL from using the index efficiently, often leading to a full table scan.

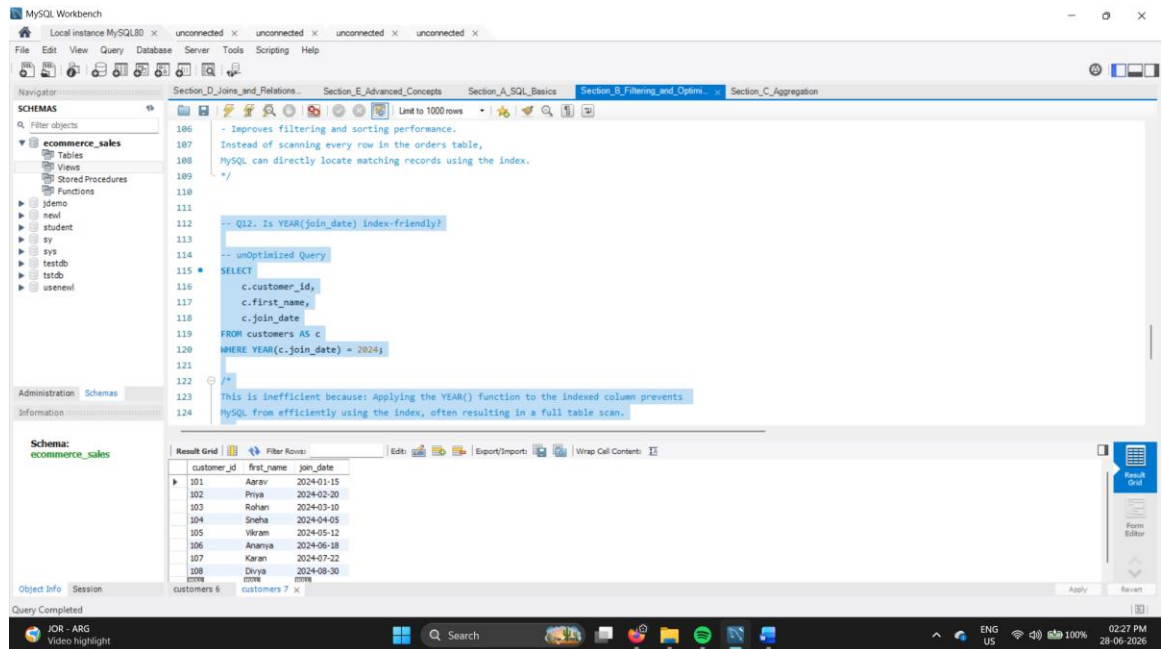
- **Unoptimized Query**

```
SELECT
  c.customer_id,
  c.first_name,
  c.join_date
FROM customers AS c
WHERE YEAR(c.join_date) = 2024;
```

- **Optimized Query**

```
SELECT
  c.customer_id,
  c.first_name,
  c.join_date
FROM customers AS c
WHERE c.join_date BETWEEN '2024-01-01'
      AND '2024-12-31';
```

- **Description:** The optimized query directly compares the join_date column with a date range, allowing MySQL to use the index effectively. This approach improves query performance and is considered a best practice.
- **Output:**

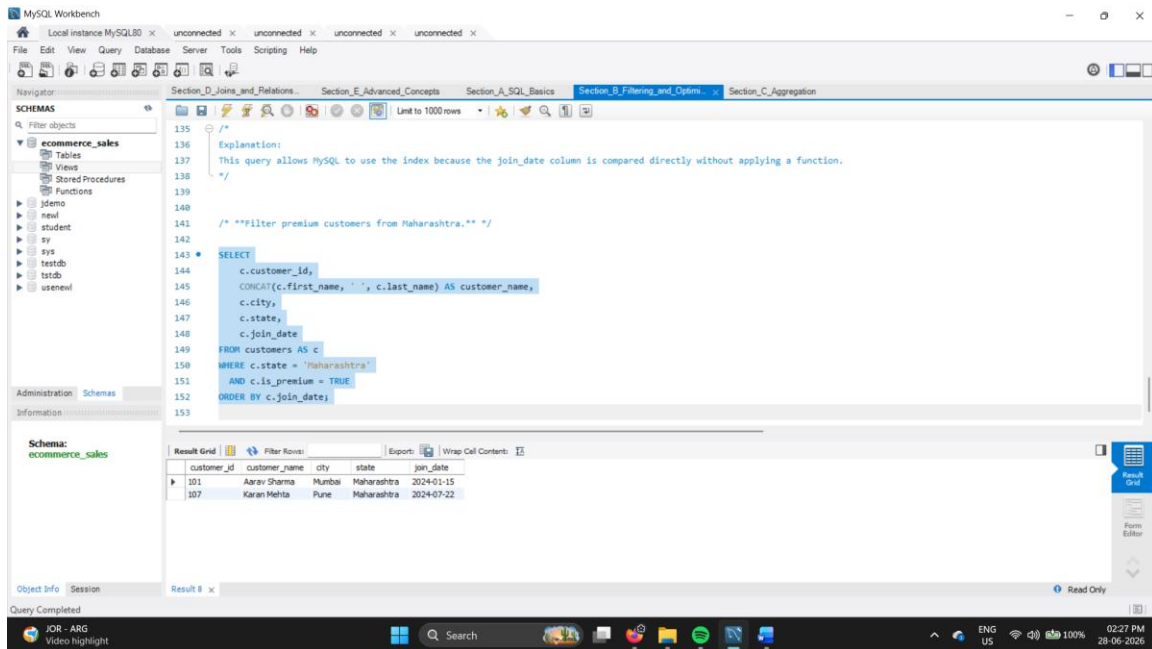


Question. Display all premium customers from **Maharashtra**.

Answer:

- **SQL Query**

```
SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    c.city,
    c.state,
    c.join_date
FROM customers AS c
WHERE c.state = 'Maharashtra'
    AND c.is_premium = TRUE
ORDER BY c.join_date;
```
- **Description:** This query retrieves premium customers residing in Maharashtra. The CONCAT() function combines the customer's first and last name into a single column, and the results are sorted by the joining date.
- **Output:**



11. Section C – Aggregation and Business Analysis

Overview: This section demonstrates the use of SQL aggregate functions to summarize and analyze business data. It includes the implementation of COUNT(), SUM(), AVG(), MIN(), MAX(), GROUP BY, HAVING, and ORDER BY to generate meaningful business insights from the e-commerce database.

Question 13. Count the total number of orders available in the orders table.

Answer:

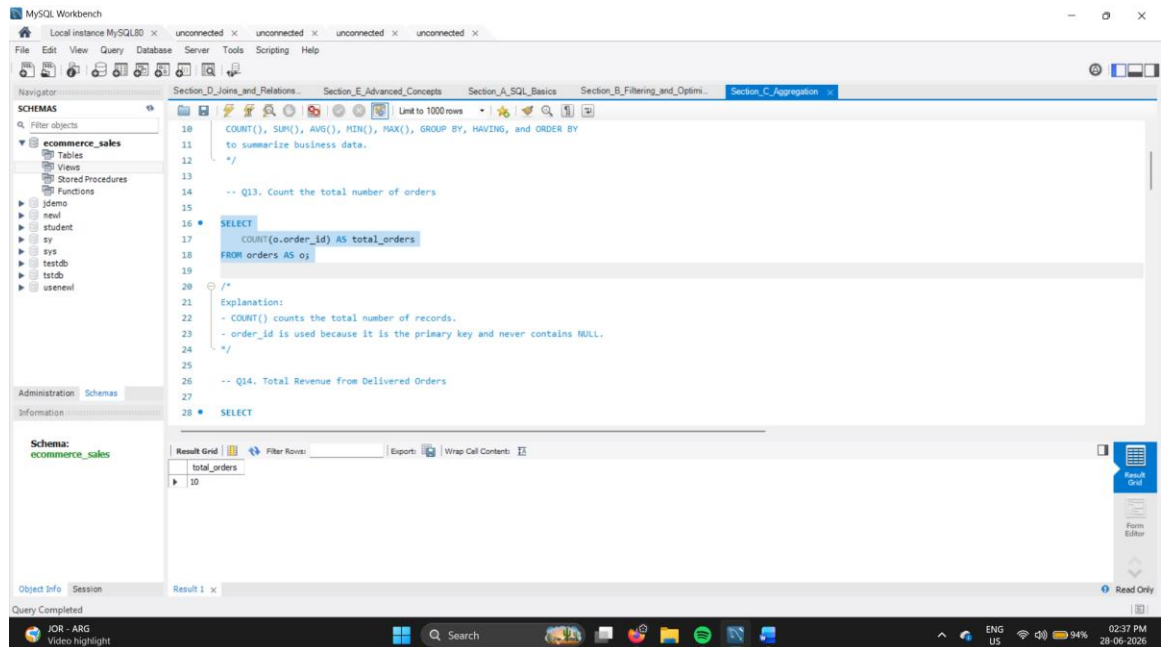
- **SQL Query**

```

SELECT
    COUNT(o.order_id) AS total_orders
FROM orders AS o;

```

- **Description:** This query counts the total number of orders present in the database using the COUNT() function. Since order_id is the primary key and cannot contain NULL values, it provides an accurate count of all order records.
- **Output:**

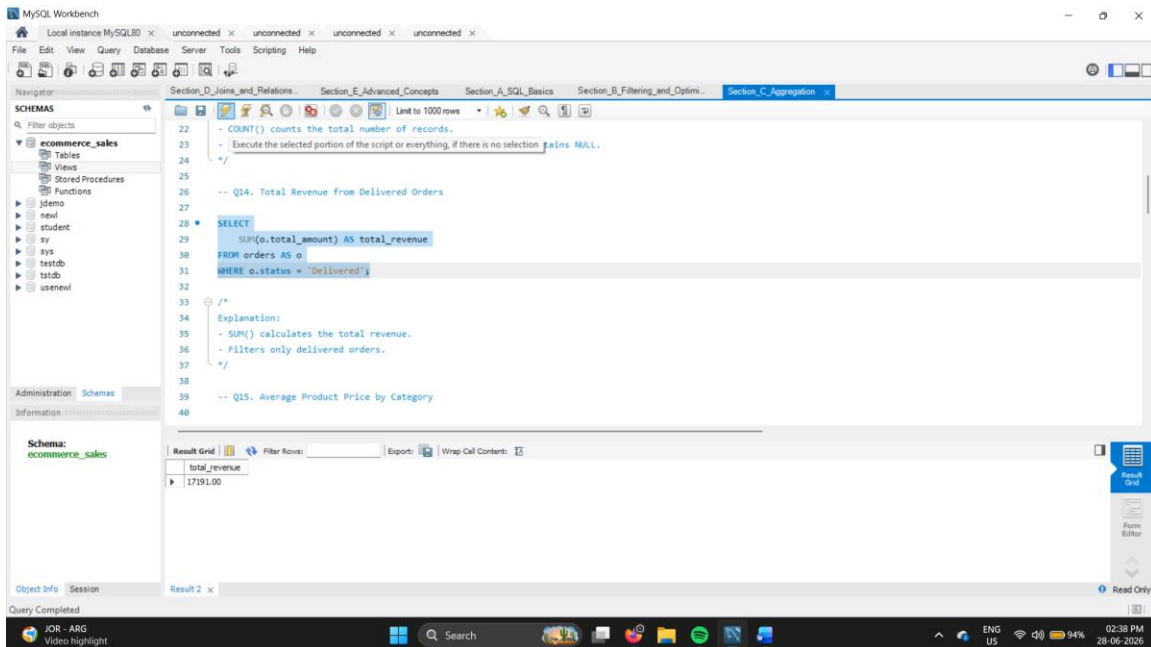


Question 14. Calculate the total revenue generated from all delivered orders.

Answer:

- **SQL Query**

```
SELECT
    SUM(o.total_amount) AS total_revenue
FROM orders AS o
WHERE o.status = 'Delivered';
```
- **Description:** This query calculates the total revenue by summing the total_amount of all orders with the status **Delivered**. It helps measure the revenue generated from successfully completed orders.
- **Output:**



Question 15. Calculate the average product price for each product category.

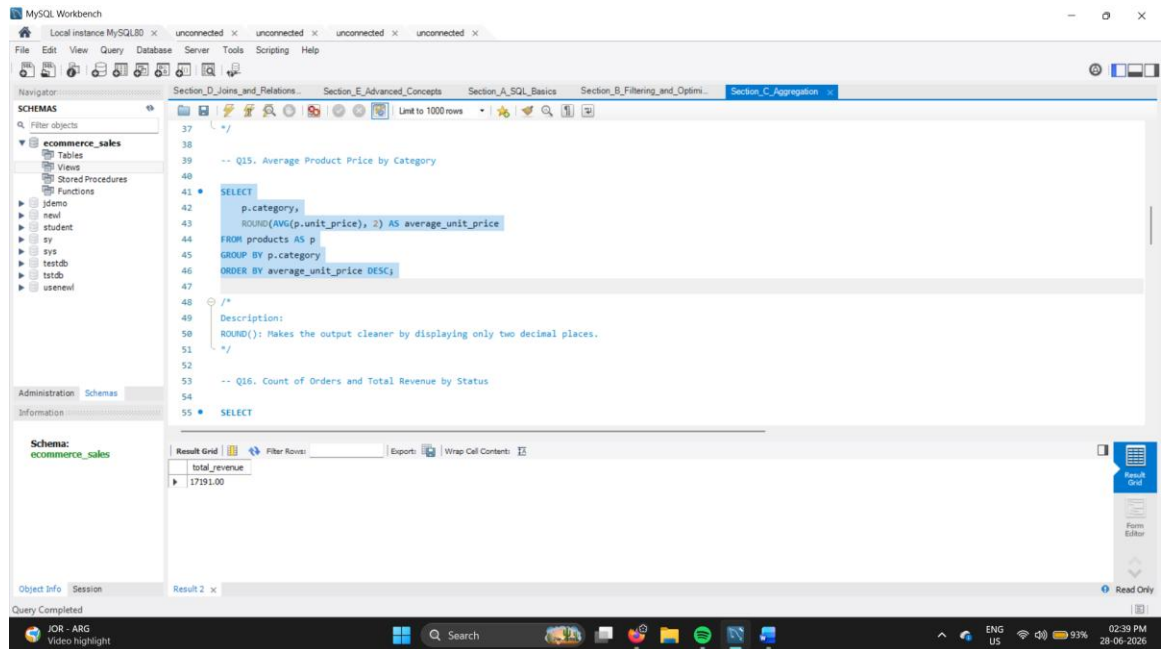
Answer:

- **SQL Query**

```

SELECT
  p.category,
  ROUND(AVG(p.unit_price), 2) AS average_unit_price
FROM products AS p
GROUP BY p.category
ORDER BY average_unit_price DESC;

```
- **Description:** This query groups products by category and calculates the average price of each category. The `ROUND()` function limits the output to two decimal places, making the results easier to read and present.
- **Output:**



Question 16. Display the total number of orders and total revenue for each order status.

Answer:

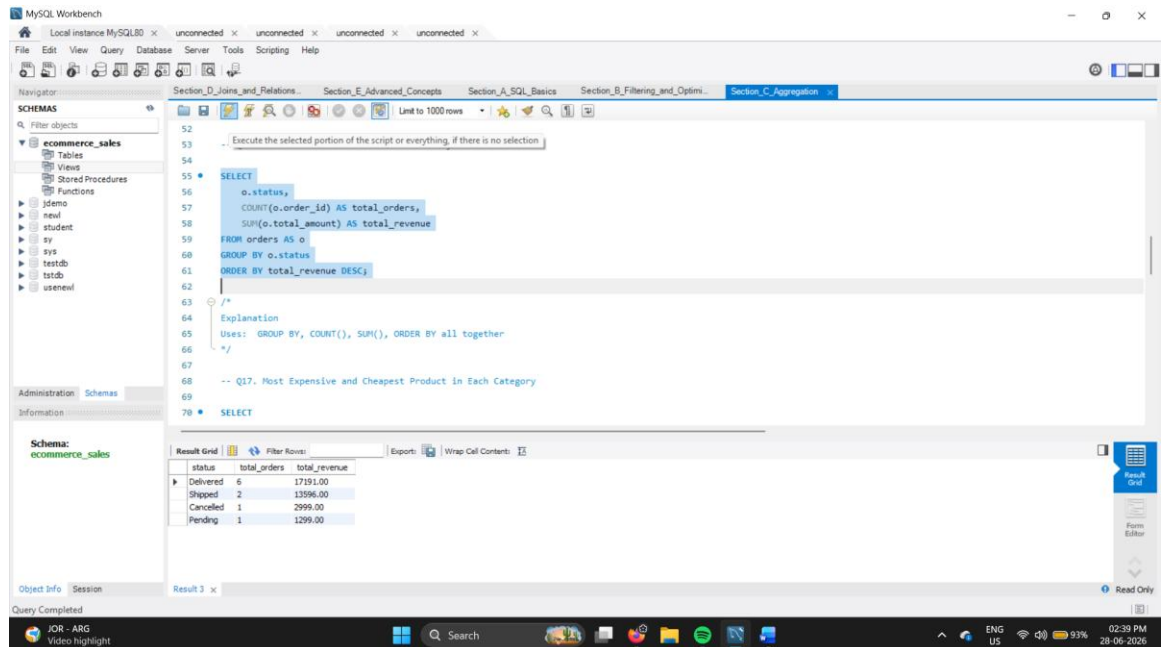
- **SQL Query**

```

SELECT
  o.status,
  COUNT(o.order_id) AS total_orders,
  SUM(o.total_amount) AS total_revenue
FROM orders AS o
GROUP BY o.status
ORDER BY total_revenue DESC;

```

- **Description:** This query summarizes order data by grouping records according to their status. It calculates both the number of orders and the total revenue for each status, helping analyze overall order performance.
- **Output:**



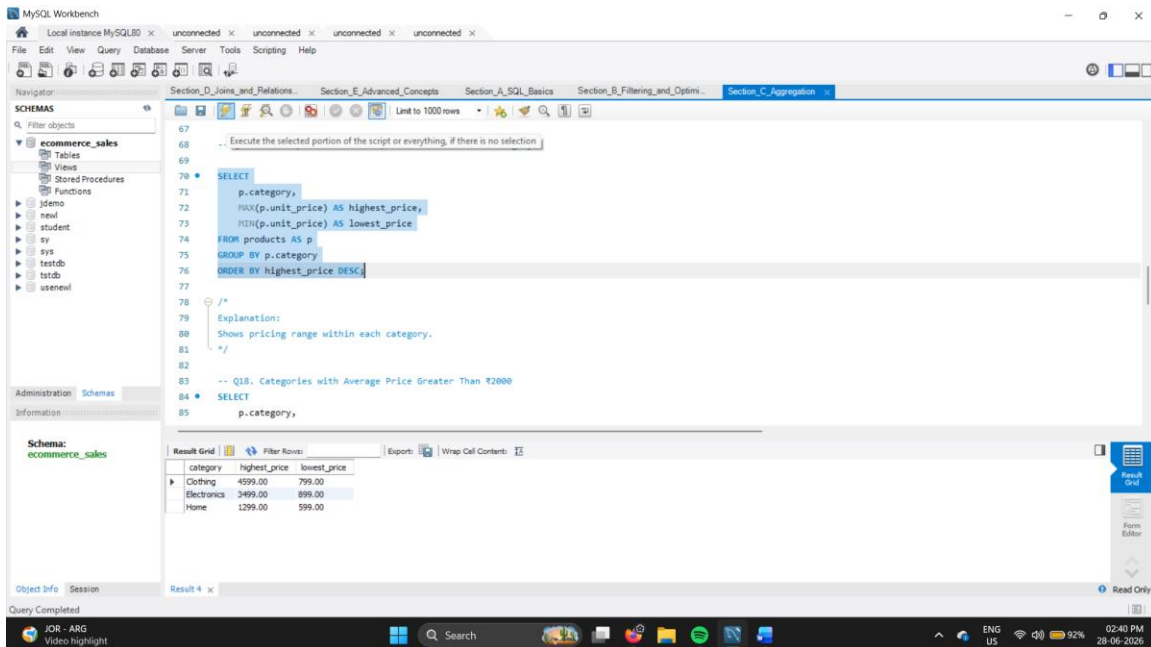
Question 17. Find the highest and lowest priced product in each category.

Answer:

- **SQL Query**

```

SELECT
  p.category,
  MAX(p.unit_price) AS highest_price,
  MIN(p.unit_price) AS lowest_price
FROM products AS p
GROUP BY p.category
ORDER BY highest_price DESC;
  
```
- **Description:** This query identifies the price range within each product category by calculating the maximum and minimum product prices. It provides a quick comparison of pricing across different categories.
- **Output:**



Question 18. Display categories whose average product price is greater than ₹2000.

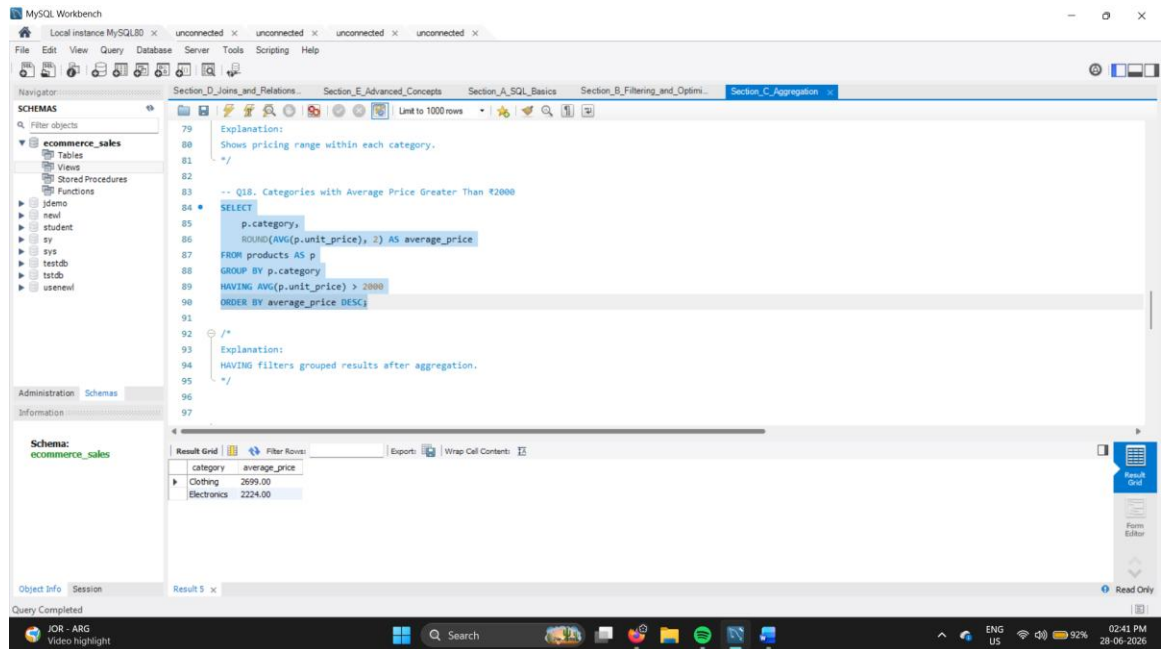
Answer:

- **SQL Query**

```
SELECT
    p.category,
    ROUND(AVG(p.unit_price), 2) AS average_price
FROM products AS p
GROUP BY p.category
HAVING AVG(p.unit_price) > 2000
ORDER BY average_price DESC;
```

- **Description:** The HAVING clause filters grouped results after aggregation. This query displays only those product categories whose average price exceeds ₹2000, making it useful for identifying premium product categories.

- **Output:**



#Question. Calculate the average discount percentage offered for each product category.

Answer:

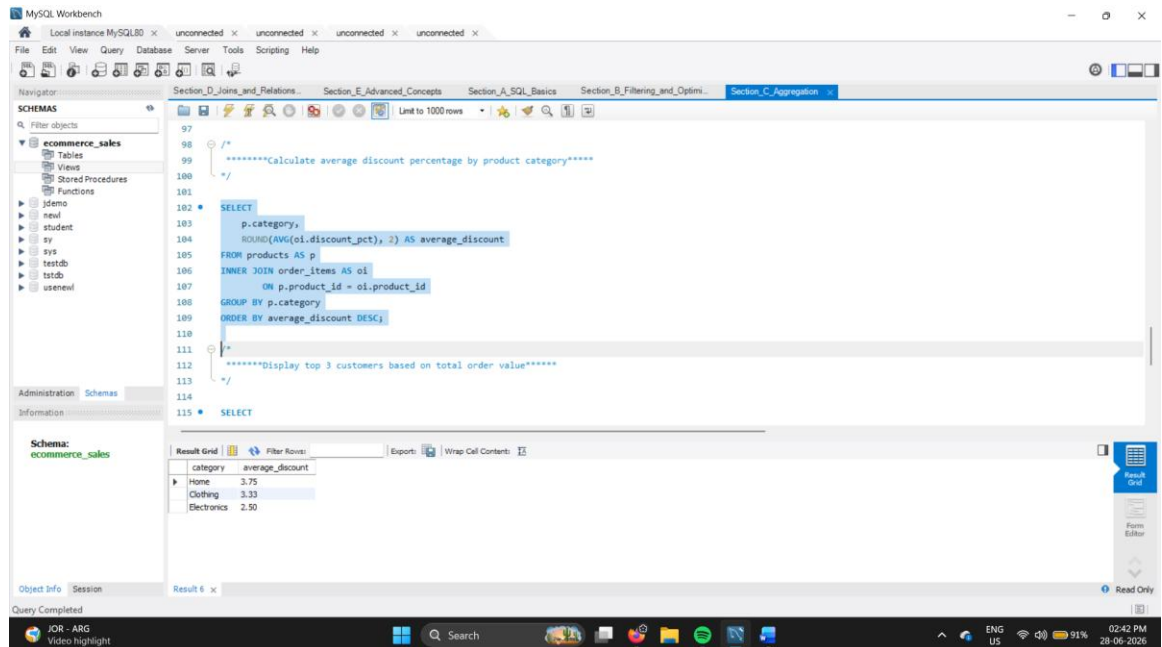
- **SQL Query**

```

SELECT
    p.category,
    ROUND(AVG(oi.discount_pct), 2) AS average_discount
FROM products AS p
INNER JOIN order_items AS oi
    ON p.product_id = oi.product_id
GROUP BY p.category
ORDER BY average_discount DESC;

```

- **Description:** This query joins the products and order_items tables to calculate the average discount offered for each product category. It helps identify categories where higher discounts are commonly applied.
- **Output:**



#Question. Display the top three customers based on their total purchase value.

Answer:

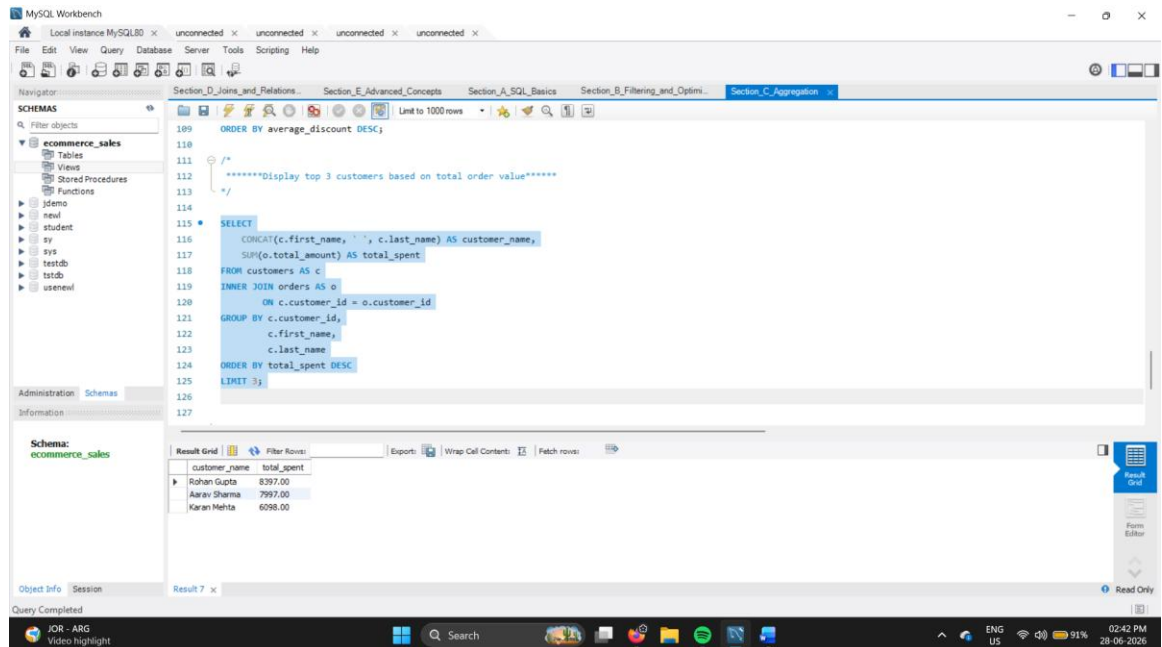
- **SQL Query**

```

SELECT
  CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
  SUM(o.total_amount) AS total_spent
FROM customers AS c
INNER JOIN orders AS o
  ON c.customer_id = o.customer_id
GROUP BY
  c.customer_id,
  c.first_name,
  c.last_name
ORDER BY total_spent DESC
LIMIT 3;

```

- **Description:** This query identifies the top three customers by calculating the total amount spent by each customer. It is useful for recognizing valuable customers and supporting customer relationship strategies.
- **Output:**



#Question. Count the number of products available in each category.

Answer:

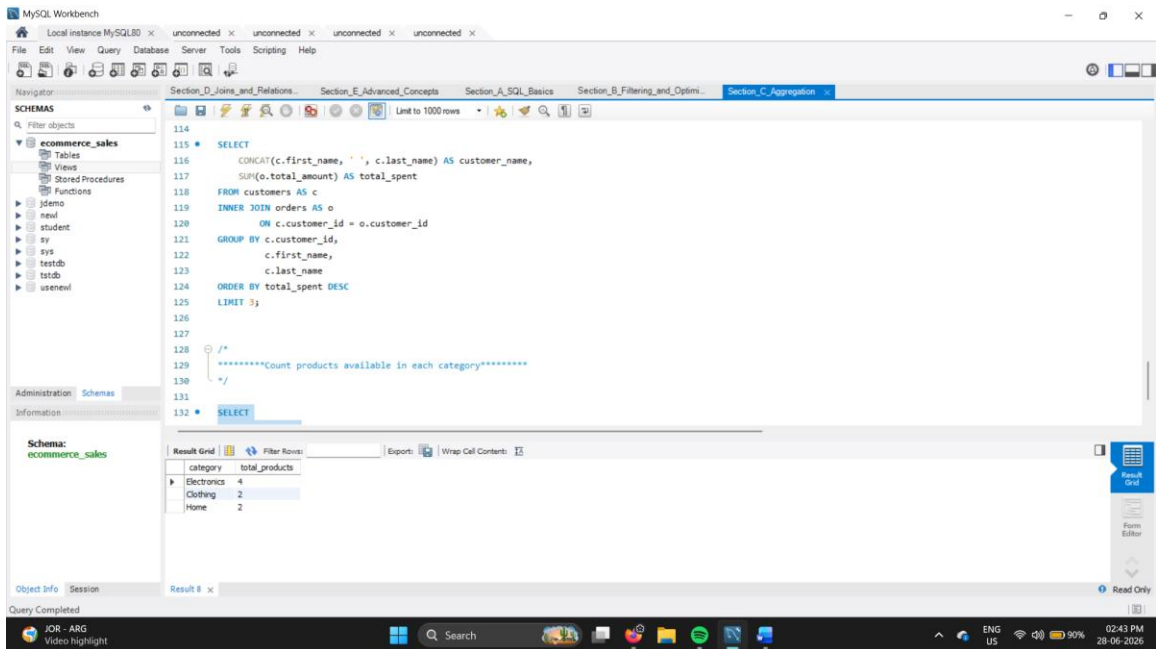
- **SQL Query:**

```

SELECT
    p.category,
    COUNT(p.product_id) AS total_products
FROM products AS p
GROUP BY p.category
ORDER BY total_products DESC;

```

- **Description:** This query counts the number of products in each category using the COUNT() function. It provides a quick overview of product distribution across different categories.
- **Output:**



12. Section D – Joins and Relationships

Overview: This section demonstrates the use of SQL joins to combine data from multiple related tables. It covers INNER JOIN, LEFT JOIN, and explains the differences between various join types. Additionally, it highlights the importance of foreign key relationships in maintaining data integrity within a relational database.

Question 19. Display order details along with customer information using an **INNER JOIN**.

Answer:

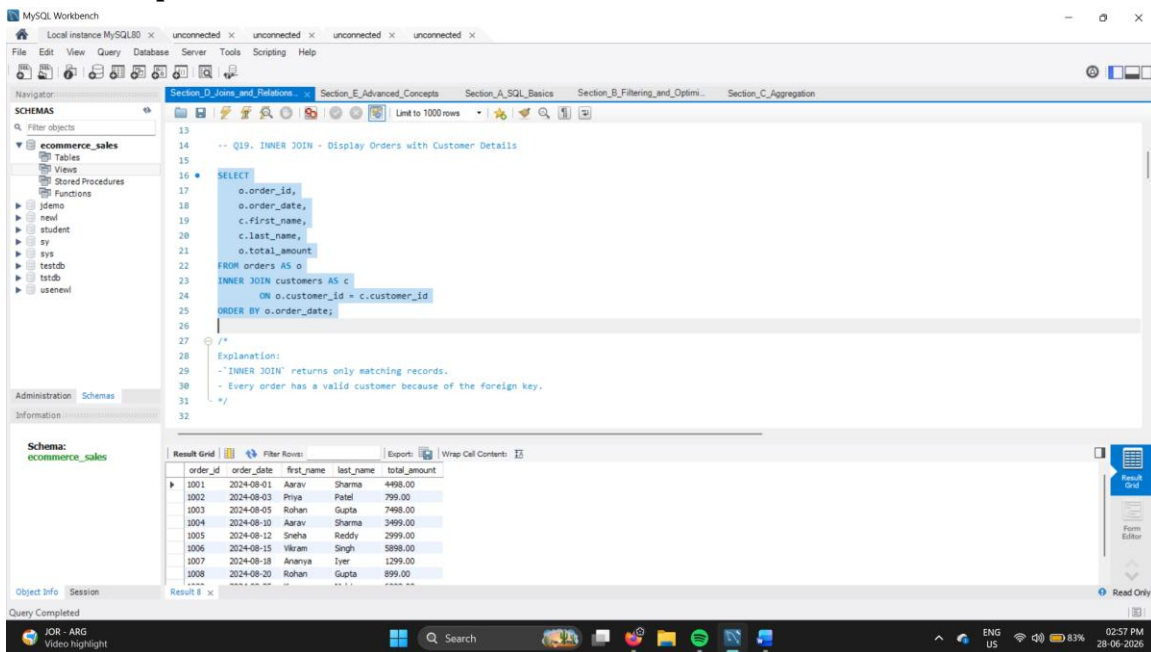
- **SQL Query:**

```
SELECT
    o.order_id,
    o.order_date,
    c.first_name,
    c.last_name,
    o.total_amount
FROM orders AS o
INNER JOIN customers AS c
    ON o.customer_id = c.customer_id
ORDER BY o.order_date;
```

- **Description**

- This query combines the orders and customers tables using an **INNER JOIN** to display customer details for each order. Since only matching records are returned, every order shown is associated with a valid customer.

- **Output:**



Question 20. Display all customers along with their orders using a **LEFT JOIN**.

Answer:

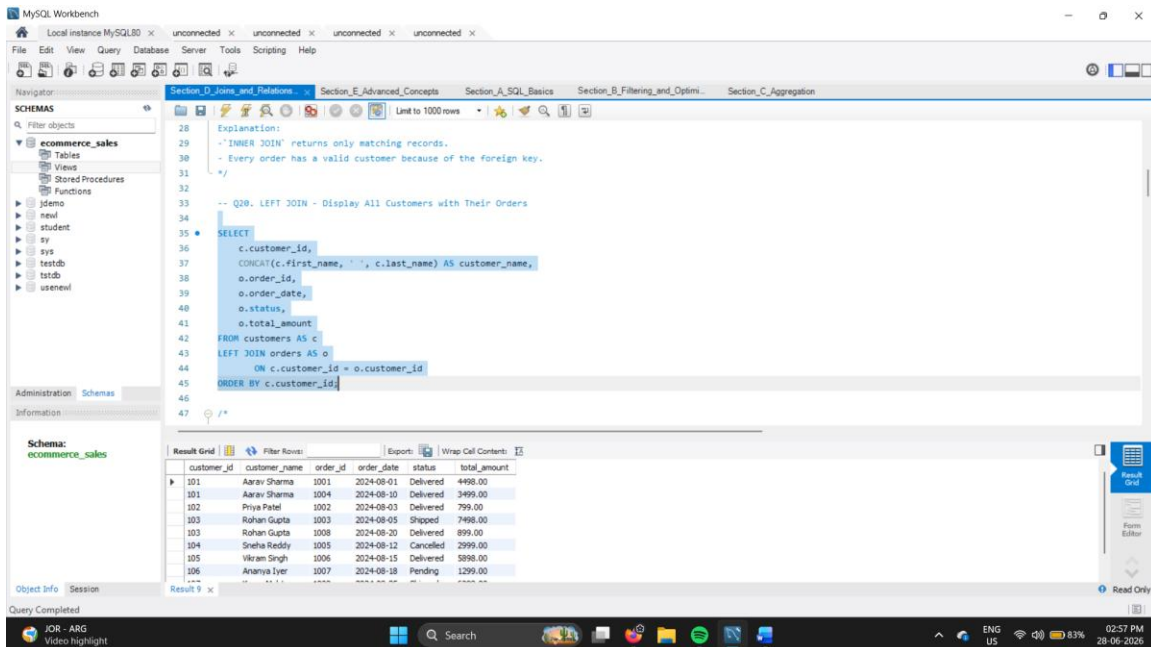
- **SQL Query:**

```

SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    o.order_id,
    o.order_date,
    o.status,
    o.total_amount
FROM customers AS c
LEFT JOIN orders AS o
ON c.customer_id = o.customer_id
ORDER BY c.customer_id;

```

- **Description:** This query retrieves all customers from the customers table, regardless of whether they have placed an order. Customers without any orders display NULL values for the order-related columns, making it useful for identifying inactive customers.
- **Output:**



Question 21. Retrieve order details, product information, and quantity by joining three related tables.

Answer:

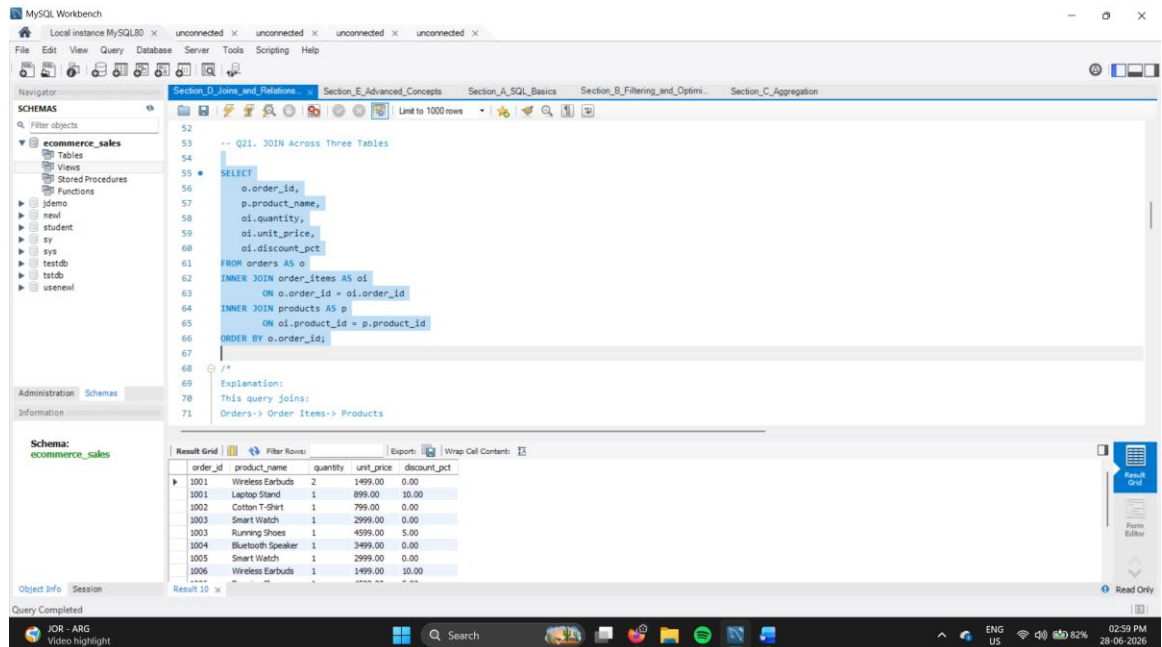
- **SQL Query**

```
SELECT
    o.order_id,
    p.product_name,
    oi.quantity,
    oi.unit_price,
    oi.discount_pct
FROM orders AS o
INNER JOIN order_items AS oi
    ON o.order_id = oi.order_id
INNER JOIN products AS p
    ON oi.product_id = p.product_id
ORDER BY o.order_id;
```

- **Description**

This query joins the orders, order_items, and products tables to display detailed information about every product included in each order. It demonstrates how multiple joins can be used to retrieve complete business information from related tables.

- **Output:**



Question 22. Explain the difference between **LEFT JOIN**, **RIGHT JOIN**, and **FULL OUTER JOIN**.

Answer:

1. LEFT JOIN

A **LEFT JOIN** returns all records from the left table along with the matching records from the right table. If no matching record exists, the columns from the right table contain NULL values.

Example

```
SELECT
  c.customer_id,
  c.first_name,
  o.order_id
FROM customers AS c
LEFT JOIN orders AS o
  ON c.customer_id = o.customer_id;
```

2. RIGHT JOIN

A **RIGHT JOIN** returns all records from the right table along with the matching records from the left table. If no matching record exists in the left table, the corresponding columns contain NULL values.

Example:

```
SELECT
  c.customer_id,
  c.first_name,
  o.order_id
FROM customers AS c
```

RIGHT JOIN orders AS o
ON c.customer_id = o.customer_id;

3. FULL OUTER JOIN

A **FULL OUTER JOIN** returns all records from both tables. Matching records are combined, while unmatched records from either table contain NULL values for the missing side.

- **Note:** MySQL does not support FULL OUTER JOIN directly. It can be simulated using the combination of LEFT JOIN, RIGHT JOIN, and UNION.
- **Description:** Each join type serves a different purpose depending on the required result. INNER JOIN returns only matching records, LEFT JOIN retains all records from the left table, RIGHT JOIN retains all records from the right table, and FULL OUTER JOIN includes all records from both tables.

Question 23. Explain the foreign key relationships used in the database.

Answer: The database contains the following foreign key relationships:

Parent Table	Child Table	Foreign Key
customers	orders	customer_id
orders	order_items	order_id
products	order_items	product_id

- **Demonstration Query**

```
INSERT INTO orders
(
    order_id,
    customer_id,
    order_date,
    status,
    total_amount
)
VALUES
(
    1015,
    999,
    '2024-09-01',
    'Pending',
    0
);
```

1500

);

- **Description:** This query attempts to insert an order with a customer ID that does not exist in the customers table. The database rejects the operation because of the foreign key constraint. This ensures referential integrity by allowing only valid customer records to be referenced in the orders table.

#Question: Display the order summary for each customer.

Answer:

- **SQL Query**

```
SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    COUNT(o.order_id) AS total_orders,
    IFNULL(SUM(o.total_amount), 0) AS total_spent
FROM customers AS c
LEFT JOIN orders AS o
    ON c.customer_id = o.customer_id
GROUP BY
    c.customer_id,
    c.first_name,
    c.last_name
ORDER BY total_spent DESC;
```

- **Description:** This query generates a customer-wise order summary by displaying the total number of orders placed and the total amount spent by each customer. The LEFT JOIN ensures that customers without any orders are also included, while IFNULL() replaces NULL values with zero.

- **Output:**

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' list with 'ecommerce_sales' selected. The main editor window contains a SQL query. The 'Result Grid' at the bottom shows the output of the query, which lists customer information and their total orders and spending.

```

/*
*****Display customer order summary*****
*/
SELECT
  c.customer_id,
  CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
  COUNT(o.order_id) AS total_orders,
  IFNULL(SUM(o.total_amount), 0) AS total_spent
FROM customers AS c
LEFT JOIN orders AS o
  ON c.customer_id = o.customer_id
GROUP BY
  c.customer_id,
  c.first_name,
  c.last_name
ORDER BY total_spent DESC;

```

customer_id	customer_name	total_orders	total_spent
103	Rohan Gupta	2	8397.00
101	Aarav Sharma	2	7997.00
107	Karan Mehta	1	6098.00
105	Vikram Singh	1	5098.00
104	Sneha Reddy	1	2999.00
108	Divya Nair	1	1598.00
106	Ananya Iyer	1	1299.00
102	Priya Patel	1	799.00

#Question. Display the total quantity sold for each product.

Answer:

SQL Query:

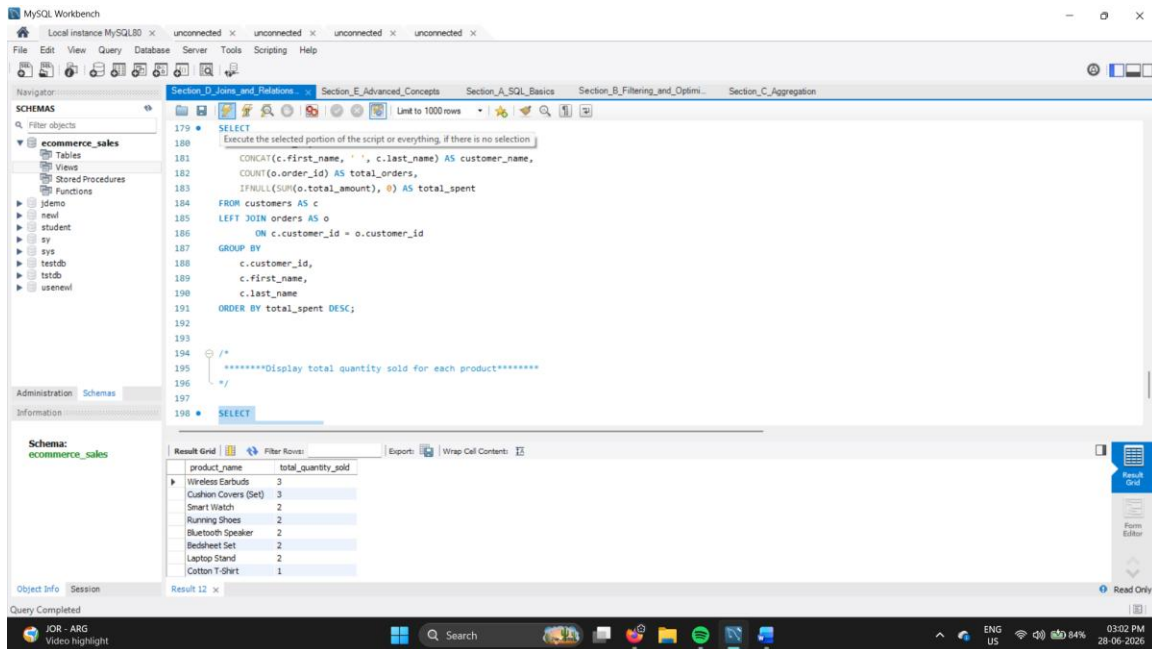
```

SELECT
  p.product_name,
  SUM(oi.quantity) AS total_quantity_sold
FROM products AS p
INNER JOIN order_items AS oi
  ON p.product_id = oi.product_id
GROUP BY p.product_name
ORDER BY total_quantity_sold DESC;

```

Description: This query calculates the total quantity sold for each product by joining the products and order_items tables. It helps identify the best-selling products and supports inventory planning and sales analysis.

Output:



13. Section E – Advanced Concepts (CASE, ACID, and Transactions)

Overview: This section demonstrates advanced SQL concepts that are widely used in real-world database applications. It includes the use of CASE statements for conditional logic, explains the ACID properties that ensure reliable database transactions, and implements transaction management using START TRANSACTION, COMMIT, and ROLLBACK to maintain data consistency

Question 24. Write a query using the CASE statement to classify products into price tiers.

- **Budget** → unit_price < 1000
- **Mid-Range** → unit_price BETWEEN 1000 AND 3000
- **Premium** → unit_price > 3000

Display the product name, unit price, and price tier.

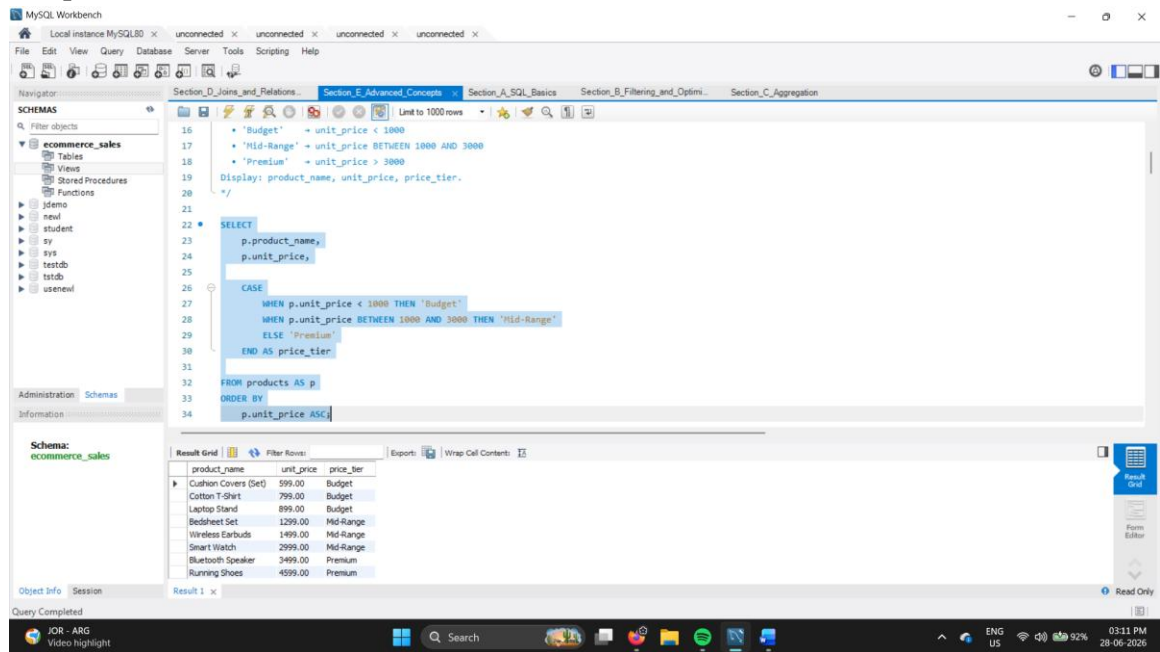
Answer:

- **SQL Query**

```
SELECT
    p.product_name,
    p.unit_price,
    CASE
        WHEN p.unit_price < 1000 THEN 'Budget'
        WHEN p.unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'
        ELSE 'Premium'
    END AS price_tier
FROM products AS p
```


ORDER BY p.unit_price ASC;

- **Description:** This query uses the CASE statement to classify products into different price categories based on their unit price. It improves the readability of reports by grouping products into meaningful business categories without modifying the original data.
- **Output:**



Question 25. Using a CASE statement inside an aggregate function, count how many orders are **Delivered** and **Not Delivered**. Display the result in a single row.

Answer:

- **SQL Query**

```

SELECT
  SUM(
    CASE
      WHEN o.status = 'Delivered'
      THEN 1
      ELSE 0
    END
  ) AS delivered_orders,

  SUM(
    CASE
      WHEN o.status <> 'Delivered'

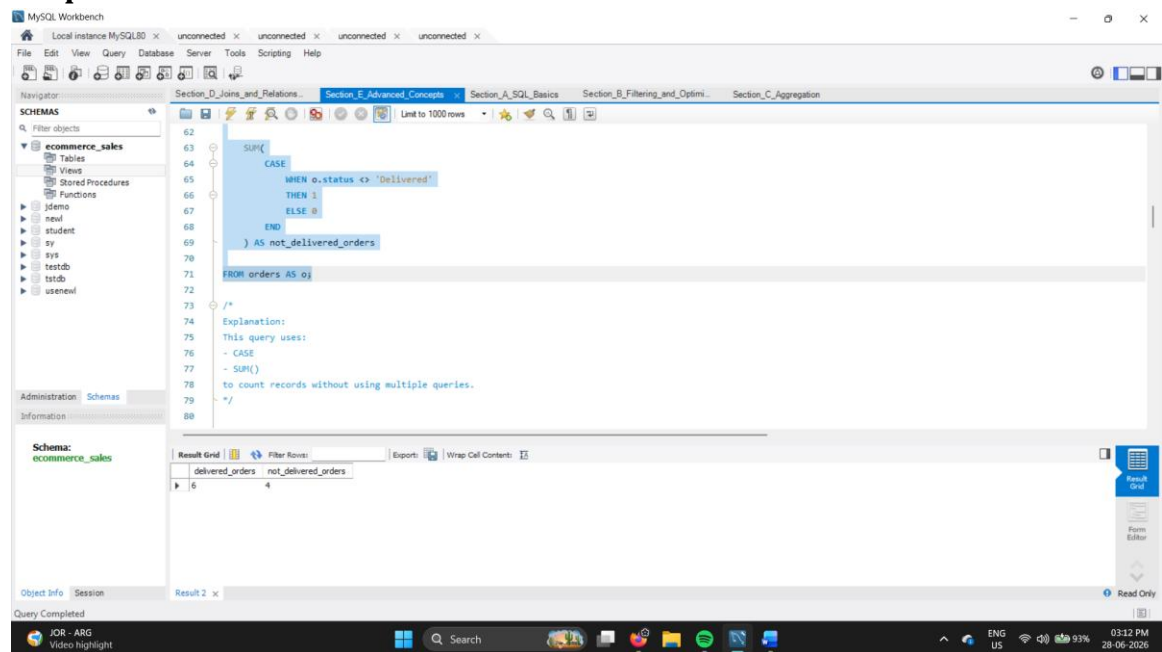
```

```

        THEN 1
        ELSE 0
    END
) AS not_delivered_orders
FROM orders AS o;

```

- **Description:** This query combines the CASE statement with the SUM() function to count delivered and non-delivered orders in a single query. It provides a concise summary of order statuses without requiring multiple SQL statements.
- **Output:**



Question 26. Explain the ACID properties of a database transaction with a real-world example.

Answer

- **A – Atomicity**

Atomicity ensures that a transaction is treated as a single unit of work. Either all operations are completed successfully, or none of them are executed.

Example: During a bank transfer, if money is deducted from one account but cannot be credited to another account, the transaction is rolled back so that no money is lost.

- **C – Consistency**

Consistency ensures that a transaction moves the database from one valid state to another while maintaining all database rules and constraints.

Example: If only five units of a product are available in stock, the database will reject an order requesting eight units to prevent invalid inventory data.

- **I – Isolation**

Isolation ensures that multiple transactions executed at the same time do not interfere with one another.

Example: If two customers attempt to purchase the last available laptop simultaneously, only one transaction succeeds while the other waits or fails, preventing duplicate sales.

- **D – Durability**

Durability guarantees that once a transaction is committed, its changes are permanently stored in the database, even if a system failure occurs immediately afterward.

Example: After a successful online payment, the payment record remains saved even if the server crashes or the power goes out.

Question 27. Write a SQL transaction that performs the following operations atomically:

1. Insert a new order.
2. Insert two order items.
3. Update the stock quantity of the purchased products.
4. Commit the transaction if all operations succeed; otherwise, roll back all changes.

Answer:

- **SQL Query**

```
START TRANSACTION;
```

```
-- Step 1: Insert a new order
```

```
INSERT INTO orders
```

```
(  
    order_id,  
    customer_id,  
    order_date,  
    status,  
    total_amount
```

```
)  
VALUES
```

```
(  
    1011,  
    102,  
    CURDATE(),  
    'Pending',  
    1598.00
```

```
);
```

-- Step 2: Insert first order item

```
INSERT INTO order_items
```

```
(  
    item_id,  
    order_id,  
    product_id,  
    quantity,  
    unit_price,  
    discount_pct
```

```
)
```

```
VALUES
```

```
(  
    16,  
    1011,  
    201,  
    1,  
    999.00,  
    0
```

```
);
```

-- Step 3: Insert second order item

```
INSERT INTO order_items
```

```
(  
    item_id,  
    order_id,  
    product_id,  
    quantity,  
    unit_price,  
    discount_pct
```

```
)
```

```
VALUES
```

```
(  
    17,  
    1011,  
    202,  
    1,  
    599.00,  
    0
```

```
);
```

-- Step 4: Update stock quantity

```
UPDATE products
```

```
SET stock_qty = stock_qty - 1
```

```
WHERE product_id = 201;
```

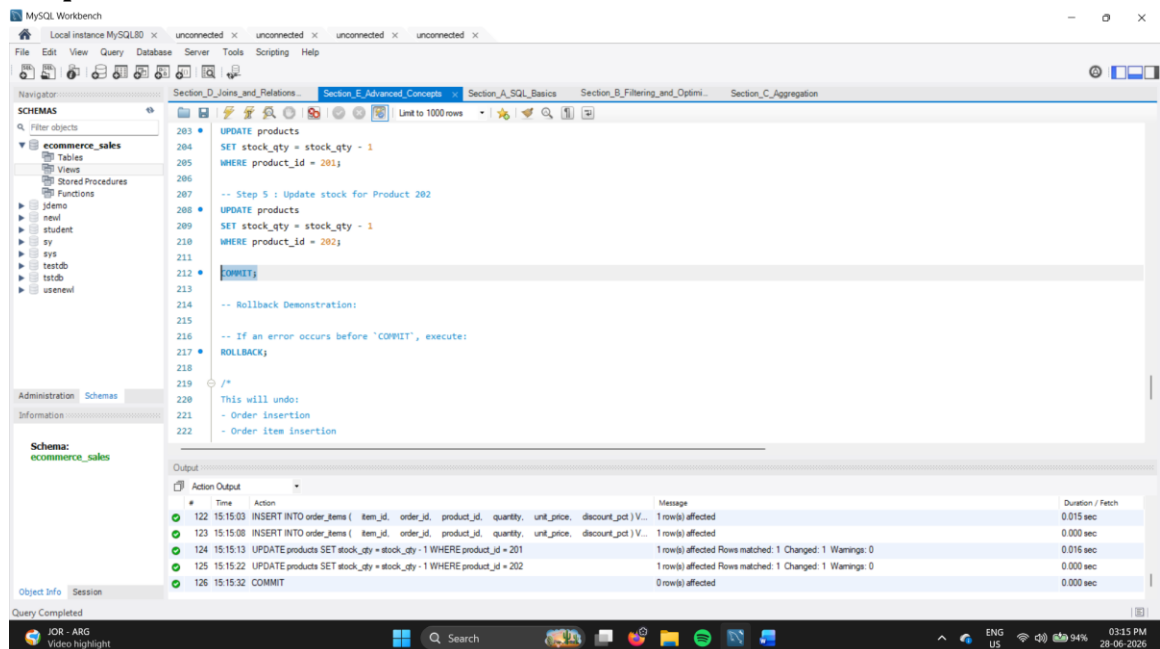
```
UPDATE products
SET stock_qty = stock_qty - 1
WHERE product_id = 202;
```

```
COMMIT;
```

```
-- If any error occurs before COMMIT,
-- execute the following statement:
```

```
ROLLBACK;
```

- **Description:** This transaction groups multiple SQL operations into a single unit of work. If every statement executes successfully, the transaction is committed permanently. If an error occurs before the commit, the ROLLBACK statement restores the database to its previous state, ensuring data consistency and preventing partial updates.
- **Output:**



#Question. Count the number of products available in each price tier.

Answer:

- **SQL Query:**

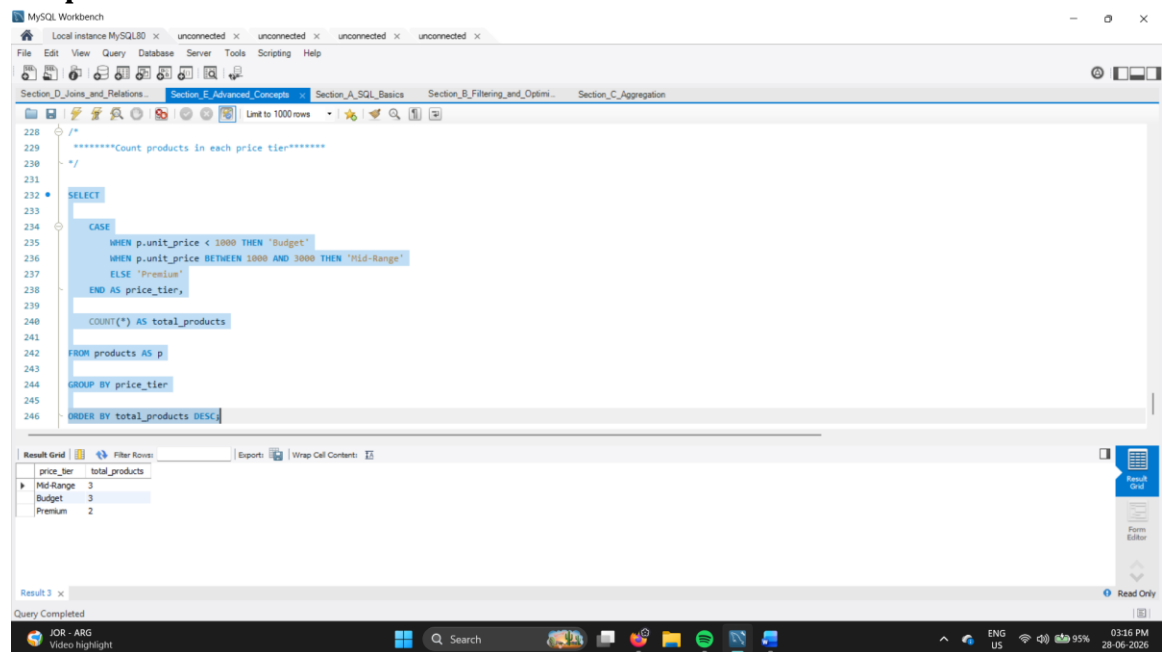
```
SELECT
CASE
WHEN p.unit_price < 1000 THEN 'Budget'
WHEN p.unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'
ELSE 'Premium'
```

```

        END AS price_tier,
        COUNT(*) AS total_products
    FROM products AS p
    GROUP BY price_tier
    ORDER BY total_products DESC;

```

- **Description:** This query categorizes products into different price tiers using the CASE statement and counts the number of products in each category. It provides a quick overview of product distribution across various pricing segments.
- **Output:**



The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```

/*
*****Count products in each price tier*****
*/
SELECT
    CASE
        WHEN p.unit_price < 1000 THEN 'Budget'
        WHEN p.unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'
        ELSE 'Premium'
    END AS price_tier,
    COUNT(*) AS total_products
FROM products AS p
GROUP BY price_tier
ORDER BY total_products DESC;

```

The Results Grid at the bottom displays the following data:

price_tier	total_products
Mid-Range	3
Budget	3
Premium	2

14. Project Business Insights:

The SQL analysis provided valuable insights into the e-commerce sales data by transforming raw records into meaningful business information. Filtering techniques helped identify specific customer segments and order statuses, while aggregation functions summarized revenue, pricing, and sales performance. Join operations combined data from multiple tables to generate comprehensive business reports, and advanced SQL concepts improved data classification and transaction reliability. Overall, the analysis demonstrates how SQL supports informed business decisions through efficient data management and reporting.

15. Learning Outcomes

During this assignment, the following concepts and skills were learned and applied:

- Designed and managed a relational database using MySQL Workbench.

- Practiced writing efficient SQL queries using filtering, sorting, aggregation, and joins.
- Applied advanced SQL concepts such as CASE statements, transactions, and ACID properties.
- Improved query performance by understanding the importance of indexes and optimization techniques.
- Developed the ability to analyze business data and generate meaningful insights from SQL query results.
- Followed professional coding standards, including query formatting, aliases, comments, and documentation.

16. Conclusion:

This assignment successfully demonstrated the practical application of SQL for analyzing e-commerce sales data. Various SQL concepts, including filtering, aggregation, joins, conditional logic, and transaction management, were implemented to solve business-oriented problems. The database was properly structured, validated, and optimized to ensure accurate and efficient query execution. Overall, this project strengthened both technical SQL skills and the ability to interpret data for business decision-making, providing a solid foundation for real-world database analysis.

17. GitHub Repository:

https://github.com/snehubhosle/CSI_Weekly_Assignments